Fused attention on one GPU

One operator, written by hand and by compiler, then measured

Balla Surya Harshith · Roll number CS23B010

CS-6023 GPU Programming · Mentor: Pravin Jagtap

Course project demo · November 2026

sequence length 2048, non-causal

4.5×

faster than the compiler's kernel

The problem, and what the two kernels achieve

THE OPERATOR

$$O = \mathrm{softmax}(Q K^{T} / \sqrt{D} + \mathrm{mask}) \cdot V$$

forward pass only, exact, fp16 in and out, fp32 state

- The naive form writes the N×N score matrix to memory. At N=2048 that is 134 MB, and it is read again for the second product.
- So the operator becomes memory-bound: measured at 31 GB/s against 170 GB/s available.
- Fixing it means fusing all three stages into one kernel, which forces the softmax to stream and forces a rescale of a live accumulator.

THE QUESTION

When does the tile model's productivity outweigh the control it gives up? Answered by measuring, not by opinion.

Correctness: both kernels pass an fp64 oracle at every size and both masks, and they agree with each other to 2.5e-04.

MEASURED, N = 2048, SIX REPETITIONS AVERAGED

4.5×

faster than cuTile

664 against 2991 µs

107%

of the measured ceiling

12.1 TFLOP/s, fp32 accumulate

1.23×

faster than cuTile, causal

364 against 448 µs

LATENCY, µs (LOWER IS BETTER)

kernel	non-causal	causal
CUDA v2 (shipped)	**664**	**364**
cuTile	2991	448
PyTorch SDPA	**617**	397
unfused baseline	13315	15098

What went wrong, and how each one was found

Two of these were errors in my own numbers. A wrong measurement is more dangerous than a slow kernel, because it points the work in the wrong direction.

What broke	What it cost	How it was found, and the fix
32-way shared-memory bank conflict	9.1 ms instead of 3.6 ms	One lane per bank, every access serialised. Padded the stride to 68/72 and interleaved the lane columns: 2.5×. Error unchanged to the last digit.
cuTile looked 1.85× faster at N=256	an unfair comparison	The tile launcher synchronised the device on every call and the CUDA path did not. Caught by comparing event timing with wall clock; the sync was removed.
Staging moved 2 bytes at a time	3.2 ms instead of 2.1 ms	SASS showed 19 scalar LDG.E.U16 sites. Replaced with 16-byte loads; SASS now shows 8 LDG.E.128. 1.53×.
My kernel was fused. cuTile's was not.	the real explanation	ncu counters: the cuTile non-causal build writes 322 MB to DRAM, mine writes 1.7 MB. Only a counter separated them; no amount of reading the source showed it.

WHAT I WOULD TELL SOMEONE STARTING THIS

Write it in cuTile first to learn whether the operator matters. Move to CUDA when a counter, not an intuition, says the remaining gap is where the accumulator lives.